Zephyr Project

IT-Schulungen.com
Technology & Transformation

GPIO-Emulator als kompakte Ereignisquelle

Technische Auflösung des Pfads aus 3.08

3.09

Ereignispfad aus 3.08 in drei Rollen

01	`Overlay`	beschreibt Emulator-Pin und Polarität.
02	`Stimulusadapter`	ändert den emulierten Eingang gezielt.
03	`Anwendung`	Callback erfasst das Ereignis. Die Verarbeitung folgt außerhalb.

Devicetree beschreibt die Verbindung. Der Emulator liefert den Stimulus. Die Anwendung verarbeitet das Ereignis.

Overlay und gpio_dt_spec

Devicetree beschreibt Port, Pin und logische Aktivität

01

Alias

event-in
verweist auf den emulierten Eingang

02

Controller

zephyr,gpio-emul
stellt GPIOs softwareseitig bereit

03

Pin

gpio_dt_spec.pin
identifiziert den Kanal

04

Polarität

GPIO_ACTIVE_LOW
trennt logische und elektrische Sicht

05

Statusausgang

optional
macht Verarbeitung beobachtbar

Anwendungscode arbeitet mit logischen Werten. Devicetree trägt die Boarddetails.

Eingang und Callback vorbereiten

INITIALISIERUNG

```
if (!gpio_is_ready_dt(&event_in)) {
    return -ENODEV;
}
int rc = gpio_pin_configure_dt(&event_in, GPIO_INPUT);
if (rc < 0) { return rc; }

gpio_init_callback(&event_cb, event_handler,
                   BIT(event_in.pin));
rc = gpio_add_callback_dt(&event_in, &event_cb);
if (rc < 0) { return rc; }

return gpio_pin_interrupt_configure_dt(
    &event_in, GPIO_INT_EDGE_TO_ACTIVE);
```

01 Bereitschaft

gpio_is_ready_dt() vor der Nutzung prüfen.

02 Konfiguration

GPIO_INPUT setzen und jeden Rückgabewert auswerten.

03 Callback

Langlebiges Objekt registrieren und danach die Flanke aktivieren.

Ein Fehler beendet die Initialisierung, bevor Ereignisse freigeschaltet werden.

Logische Zustände und aktive Polarität

LOGISCHER WERT

Anwendungssicht

gpio_pin_get_dt() liefert 0 oder 1 nach Berücksichtigung der Devicetree-Flags.

AKTIVE POLARITÄT

Devicetree entscheidet

GPIO_ACTIVE_LOW macht einen niedrigen Pegel logisch aktiv.

PORTABILITÄT

Gleicher Anwendungscode

Boarddetails bleiben in Overlay und gpio_dt_spec.

TO_ACTIVE beschreibt den logischen Aktivzustand, nicht zwingend eine steigende elektrische Flanke.

gpio_emul_input_set_dt() als Test-Stimulus

EMULATOR-API

```
#include <zephyr/drivers/gpio/gpio_emul.h>

int rc = gpio_emul_input_set_dt(&event_in, 1);
if (rc < 0) {
    shell_error(sh, "stimulus failed: %d", rc);
    return rc;
}
```

01 Externer Eingang

Die API ändert den Wert des emulierten Eingangspins.

02 Reproduzierbar

Feste Folgen erzeugen definierte Zustandswechsel.

Der Stimulus stammt aus Software, durchläuft danach aber den normalen GPIO-Pfad der Anwendung.

Callback-Grenze und Verarbeitungskontext

GPIO-CALLBACK

Minimal erfassen

Pinmaske prüfen

Zähler oder Timestamp erfassen

Ohne Warten signalisieren

WORKQUEUE

Fachlogik ausführen

Zustand aktualisieren

Logs erzeugen

Statusausgang setzen

SHELL ODER TEST

Stimulus auslösen

```
Emulator-API aufrufen
Status und Statistik lesen
Ablauf reproduzieren
```

Callback, Verarbeitung und Bedienung bleiben getrennte Verantwortungen.

gpio set ist kein Eingangsstimulus

ZWEI UNTERSCHIEDLICHE ABSICHTEN

```
gpio_pin_set_dt(&status_out, 1);
// Anwendung treibt einen Ausgang


gpio_emul_input_set_dt(&event_in, 1);
// Testumgebung ändert einen Eingang
```

01 **Ausgang treiben**

gpio set bedient einen als Ausgang konfigurierten Pin.

02 **Eingang stimulieren**

gpio_emul_input_set_dt() injiziert einen externen Zustand in den Emulator.

Ausgangssteuerung und Eingangsstimulus verfolgen unterschiedliche Richtungen im Signalfluss.

Aussagekraft, Grenzen und Lernkontrolle

BELEGT

Funktionaler Pfad

Overlay wird ausgewertet

Callback wird ausgelöst

Verarbeitung wird beobachtbar

NICHT BELEGT

Reale Hardware

Elektrische Eigenschaften

Kontaktprellen und Störungen

Reale Interruptlatenz

ERKLÄREN

Vollständige Kette

Emulatoraufruf

GPIO-Callback und Signalisierung

Verarbeitung außerhalb des Callbacks

native_sim prüft Logik und Ereignisfluss. Elektrik und Timing werden später optional am Zielsystem geprüft.